

INTERFACING ADC USING 8051 MICROCONTROLLER:

In this article you will learn how to interface, analog to digital converter with 8051 microcontroller. Unlike, pic microcontroller, Arduino and avr microcontroller, 8051 microcontroller do not have built in ADC. if we want to interface any sensor with 8051 microcontroller, we have to use external ADC. For example, you want to measure temperature with 8051 microcontroller and you are using LM35 temperature sensor to measure temperature. LM35 temperature sensor gives output in the form of analog voltage. So we need to use analog to digital converter. We use Analogue to digital convertor (ADC) to convert the analogue signal into digital form. Analogue signal can be the output of some sensor. And then the data in digital format can then be used for further processing by the digital processors. Now in this tutorial we will learn about ADC 0804 and its interfacing using 8051 microcontroller.

FEATURES OF ADC 0804:

Its main features are given below:

8 bit resolution

Differential analogue voltage inputs

0-5V input voltage range

No zero adjustment

Built-in clock generator

Voltage at $V_{ref}/2$ (pin9) can be externally adjusted to convert smaller input voltage spans to full 8 bit resolution.

RESOLUTION:

Resolution refers to the conversion of an analog voltage to a digital value. It indicates the number of discrete values that an ADC can produce over the range of analog values. The values are usually stored electronically in binary form, so the resolution is usually expressed in bits.

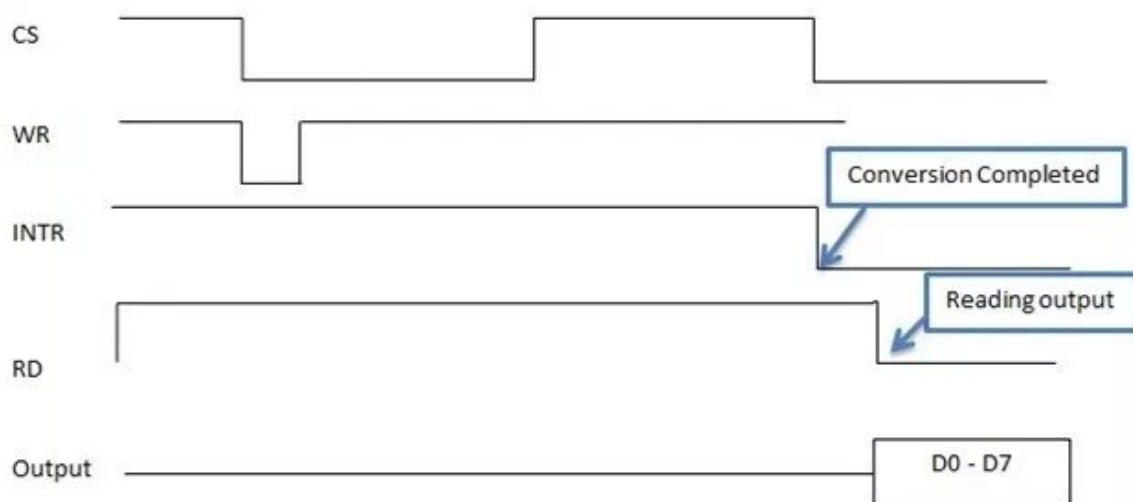
STEP SIZE:

It is the voltage difference between one digital level and the next level that can be measured by ADC. In a 4-bit converter, an input of 1 Volt produce an output of 0001, then the step size is 1 Volt. 0 volts is always considered 0000. Distortion in the signal can

be reduced by decreasing the step size to 0.5 volts but it lowers the maximum input reading. If we are using smaller step size, higher bit converter is needed to have maximum voltage.

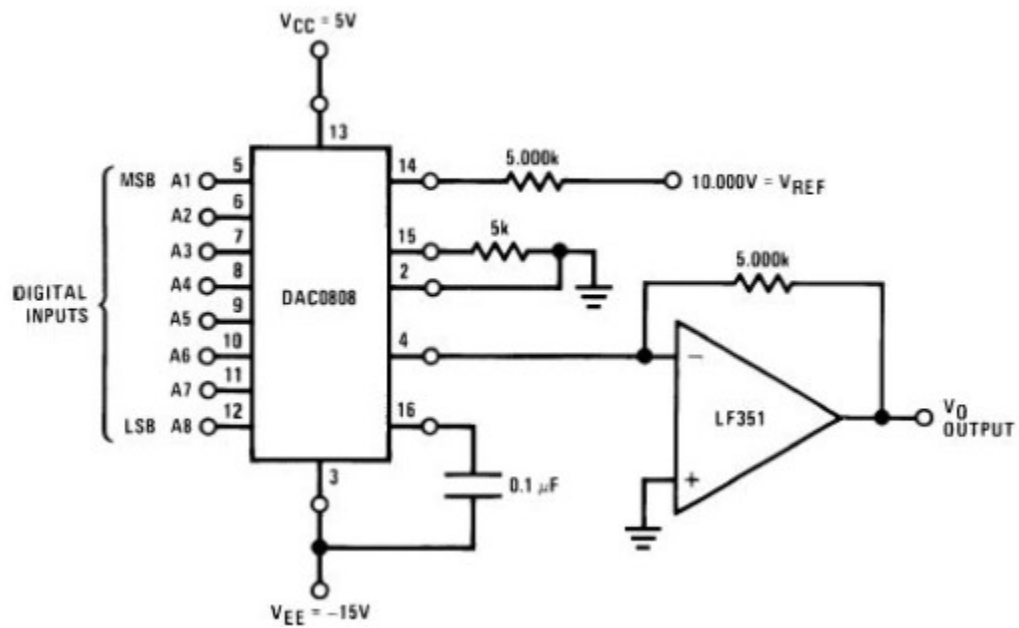
ADC0804 is a single channel analog to digital converter. It takes only one analog signal. It has 8 bit resolution. Other ADC can have n bit resolution and n can be 8,10,12,16 or 24 bits. If ADC has higher resolution, it gives smaller step size. If ADC has 8 bit resolution, input voltage span is 0-5V and the step size is 19.53mV ($5V/255$). In this situation we called $V_{ref}/2$ as left open.

As we know that microcontroller will provide control signals to the ADC. Its main connections are as follow: ADC0804 conversion steps

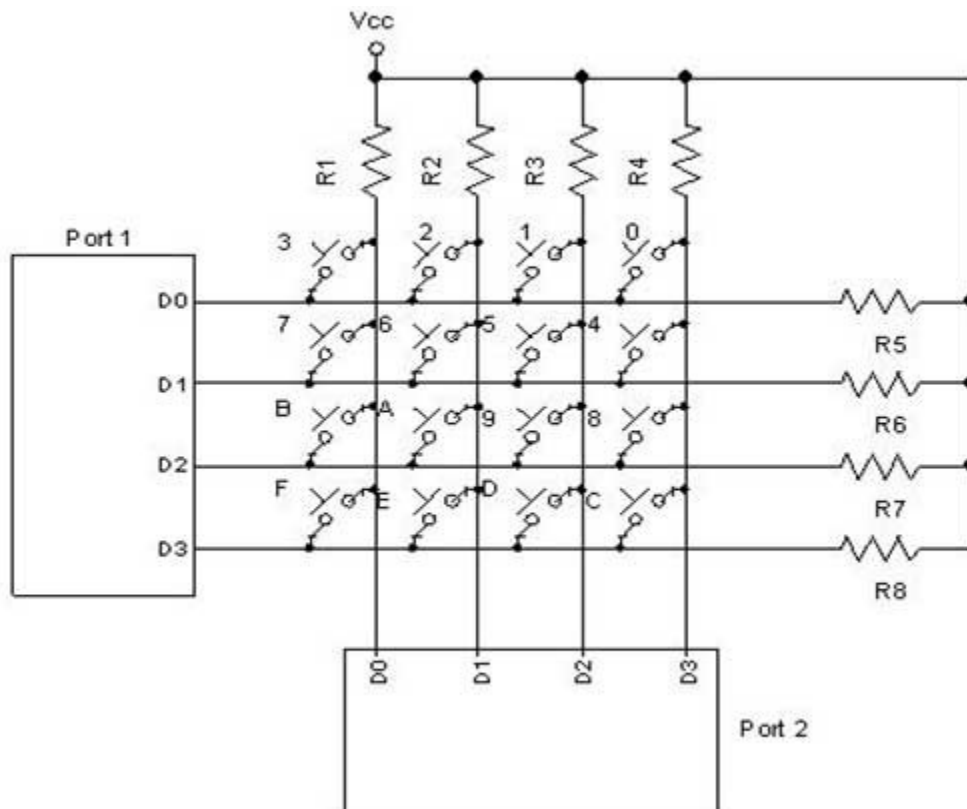


Interfacing DAC with 8051

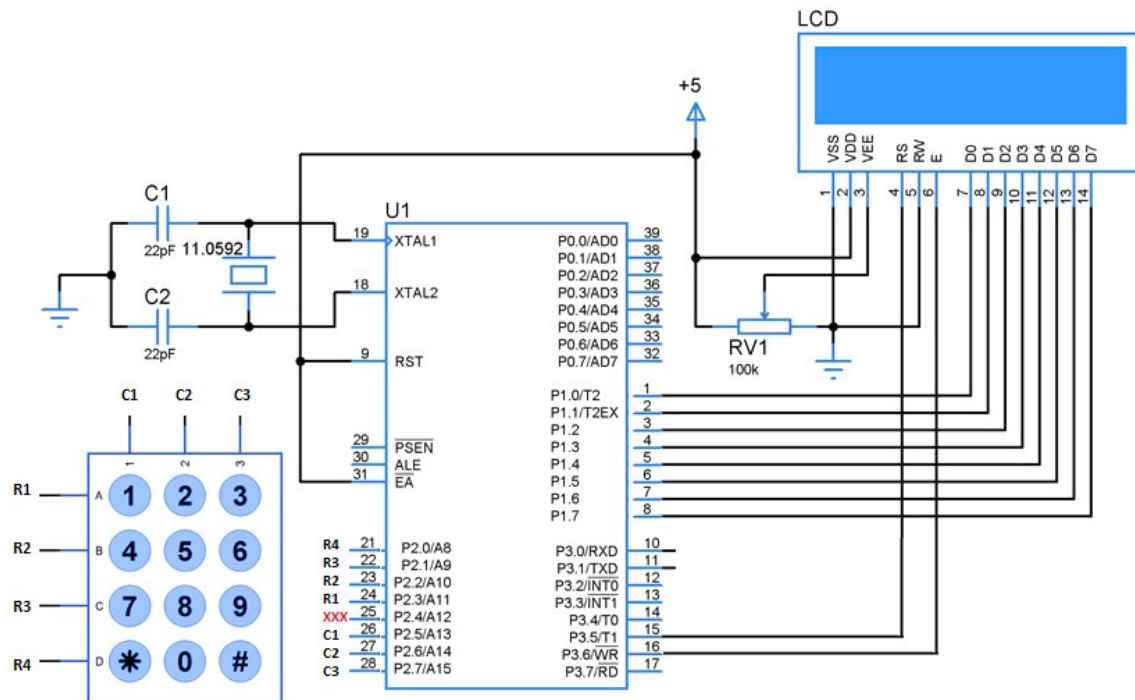
For generation of waveform i was using 8080 DAC which is an 8 bit DAC with current output. The output of the DAC is given to the op amp which works as current to voltage converter.



Interfacing 4x4 keyboard with 8051



Interface 16x2 LCD with 8051



```
#include<reg52.h>
```

```
#include<stdio.h>
```

```
#include<string.h>
```

```
#include "LCD_8_bit.h" /* 16x2 LCD header file*/
```

```
#define keyport P1
```

```
sbit RS = P3^5; /* RS(register select) for LCD16x2 */
```

```
sbit RW = P3^6; /* RW(Read/write) for LCD16x2 */
```

```
sbit ENABLE = P3^7; /* EN(Enable) pin for LCD16x2*/
```

```

unsigned char keypad[4][4] = { {'7','8','9','/'},
                                {'4','5','6','x'},
                                {'1','2','3','-'},
                                {' ','0','=','+'} };

```

```

unsigned char colloc, rowloc;

```

```

unsigned char key_detect()

```

```

{
    keyport=0xF0;          /*set port direction as input-output*/
    do
    {
        keyport = 0xF0;
        colloc = keyport;
        colloc&= 0xF0;     /* mask port for column read only */
    }while(colloc != 0xF0); /* read status of column */

    do
    {
        do
        {
            delay(20);     /* 20ms key debounce time */
            colloc = (keyport & 0xF0); /* read status of column */
        }while(colloc == 0xF0); /* check for any key press */
    }
}

```

```

        delay(1);

        colloc = (keyport & 0xF0);
    }while(colloc == 0xF0);

while(1)
{
    /* now check for rows */

    keyport= 0xFE;
        /* check for pressed key in 1st row */

    colloc = (keyport & 0xF0);

    if(colloc != 0xF0)
    {

        rowloc = 0;

        break;

    }


    keyport = 0xFD;
    /* check for pressed key in 2nd row */

    colloc = (keyport & 0xF0);

    if(colloc != 0xF0)
    {

        rowloc = 1;

        break;

    }

```

```

keyport = 0xFB;                                /* check for pressed key in 3rd row */
colloc = (keyport & 0xF0);
if(colloc != 0xF0)
{
    rowloc = 2;
    break;
}

keyport = 0xF7;                                /* check for pressed key in 4th row */
colloc = (keyport & 0xF0);
if(colloc != 0xF0)
{
    rowloc = 3;
    break;
}
}

if(colloc == 0xE0)
{
    return(keypad[rowloc][0]);
}

else if(colloc == 0xD0)
{
    return(keypad[rowloc][1]);
}

```

```

    }
    else if(colloc == 0xB0)
    {
        return(keypad[rowloc][2]);
    }
    else
    {
        return(keypad[rowloc][3]);
    }
}

int main(void)
{
    LCD_Init();
    LCD_String_xy(1,0,"Press a key");

    while(1){
        LCD_Command(0xc0);
        LCD_Char(key_detect());        /* Display which key is pressed */
    }
}

```